

Using an LLM to generate Instruction Following examples

In the module on [Instruction Following \(LLM Instruction Following.ipynb\)](#)

- we motivated the use of Fine-Tuning a LLM
- to exhibit Instruction Following behavior

Recall: an example of Instruction Following behavior is a triple

$\langle \text{Instruction, Context, Response} \rangle$

for example

- Instruction: "Tell me the word that is the opposite of the word that I input"
- Context: "Input: Stop"
- Response: "Go"

The Instruction describes the task to be accomplished

- relationship between Input and Response
- the Input/Response pair is an exemplar for this task

In this module, we explore methods

- to generate these fine-tuning examples
- to improve examples

Using an LLM to generate Instruction Following examples

Reference

[SELF-Instruct paper \(https://arxiv.org/pdf/2212.10560.pdf\)](https://arxiv.org/pdf/2212.10560.pdf)

Is there an alternative to the labor-intensity of constructing Instruction Following examples by human ?

The answer is "Yes"

- and involves the use of In-Context learning.

We can imagine the process as

- starting with a small number k of human-constructed examples
$$\langle \text{Instruction}^{(1)}, \text{Context}^{(1)}, \text{Response}^{(1)} \rangle$$
$$\vdots$$
$$\langle \text{Instruction}^{(k)}, \text{Context}^{(k)}, \text{Response}^{(k)} \rangle$$
- which are used as exemplars in a *few-shot* learning prompt

the hope is that the LLM will infer

- given a non-exemplar prompt (one without a Response)
$$\langle \text{Instruction}^{(k+1)}, \text{Context}^{(k+1)}, \rangle$$

it's desired completion adds the Response

$$\langle \text{Instruction}^{(k+1)}, \text{Context}^{(k+1)}, \text{Response}^{(k+1)} \rangle$$

But

- where does a wide variety of Instructions come from ?
- given an Instruction: where do the Context and Response come from ?

The actual process

- is multi-step
- using In-Context learning for each step.

The initial data available

- is a **small** set of "seed" examples
- human generated, perhaps
 ⟨Instruction, Context, Response⟩

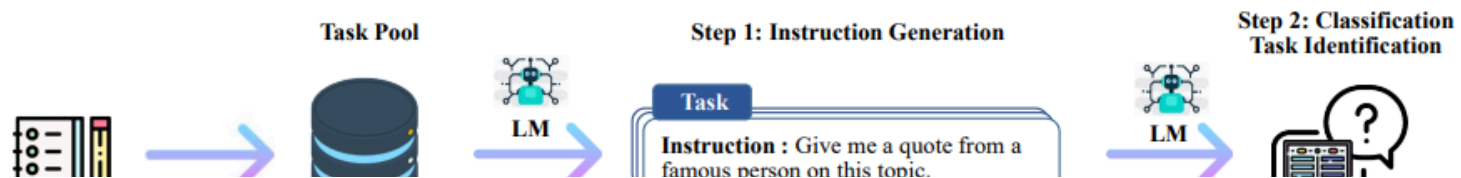
The seed data

- is first used to augment the set of possible Instructions
- and then used to generate the Context and Response

resulting in new synthetic triples

Here is an overview of the method

- we will reference this diagram in the following sub-sections



Generating the Instruction part of an Instruction-Output example

The first step is to generate the first Instruction part (i.e., the Instruction) of the triple

$$\langle \mathbf{Instruction}^{(k+1)}, \text{Context}^{(k+1)}, \text{Response}^{(k+1)} \rangle$$

using k exemplars

$$\begin{array}{c} \langle \text{Instruction}^{(1)} \rangle \\ \vdots \\ \langle \text{Instruction}^{(k)} \rangle \end{array}$$

See the box labeled "Step 1" in the illustration above

The exemplars are the "seed" data in the diagram.

Here is the template for the exemplars used to generate Instructions.

B Prompting Templates for Data Generation

SELF-INSTRUCT relies on a number of prompting templates in order to elicit the generation from language models. Here we provide our four templates for generating the instruction (Table 6), classifying whether an instruction represents a classification task or not (Table 7), generating non-classification instances with the input-first approach (Table 8), and generating classification instances with the output-first approach (Table 9).

```
Come up with a series of tasks:

Task 1: {instruction for existing task 1}
Task 2: {instruction for existing task 2}
Task 3: {instruction for existing task 3}
Task 4: {instruction for existing task 4}
Task 5: {instruction for existing task 5}
Task 6: {instruction for existing task 6}
Task 7: {instruction for existing task 7}
Task 8: {instruction for existing task 8}
Task 9:
```

Table 6: Prompt used for generating new instructions. 8 existing instructions are randomly sampled from the task pool for in-context demonstration. The model is allowed to generate instructions for new tasks, until it stops its generation, reaches its length limit or generates “Task 16” tokens.

Attribution: <https://arxiv.org/pdf/2212.10560.pdf#page=15>
(<https://arxiv.org/pdf/2212.10560.pdf#page=15>).

With the above template, we expect the LLM

- to generate a continuation of the prompt
 - ending with Task 9:
- which is the Instruction part of a new task

Generating the Context/Response (Input/Output) part, given an Instruction

Once we have generated a the Instruction part

$\text{Instruction}^{(k+1)}$

of the new synthetic example, we need to generate the Context and Response

Using the augmented set of Instructions

- seed data + synthetic Instructions

we use exemplars

$$\begin{aligned} &\langle \text{Instruction}^{(1)}, \text{Context}^{(1)}, \text{Response}^{(1)} \rangle \\ &\vdots \\ &\langle \text{Instruction}^{(k)}, \text{Context}^{(k)}, \text{Response}^{(k)} \rangle \end{aligned}$$

and prompt

$$\text{Instruction}^{(k+1)}$$

with the expectation that the LLM's continuation will be

$$\text{Context}^{(k+1)}, \text{Response}^{(k+1)}$$

See the box labeled "Step 3" in the diagram above

To illustrate:

For Classification tasks, the prompt might look like this

Task: Classify the sentiment of the sentence into positive, negative, or mixed

Example 1

Sentence: I enjoy the flavor of the restaurant but their service is too slow.

Class Label: mixed

Example 2

Sentence: I had a great day today. The weather was beautiful and I spent time with friends.

Class label: Positive

Task: Tell me if the following email is a promotion email or not.

Email: Check out our amazing new sale! We've got discounts on all of your favorite products.

Class label: Promotion

Email: We hope you are doing well. Let us know if you need any help.

Class label: Not Promotion

The last line above contains a place holder for the Instruction of the Target Task

Instruction^($k+1$)

that we created in Step 1.

Here is an example of the template from the paper

```
Task: Suggest a better and more professional rephrasing of the following sentence.
Example 1
Sentence: This house is surprisingly not constructed very well, and you probably need more
money to fix it after you buy it. If you ask me, I would suggest you to consider other
candidates.
Output: This house does not seem to be constructed well, so you may need to spend more money
to fix it after you purchase it. I would suggest that you look at other properties.
Example 2
Sentence: Just so you know, we did an experiment last week and found really surprising results
- language model can improve itself!
Output: Our experiments last week demonstrated surprising results, proving that the language
model can improve itself.
...

Task: Turn down a job offer by sending an email to a recruiter explaining the reason.
Output: Hi [Recruiter],
Thank you so much for the generous offer to join your team. As we discussed, I've admired the
company for a number of years, and am a proud endorser of its products. However, after further
consideration of where I currently am in my career, I've decided to accept an offer at another
company.
I would love to stay in touch with you and have already started following you on [Social Media
Platform]. Again, thank you so much for your time and consideration.
Thanks again,
[Your Name]

Task: {Instruction for the target task}
```

Table 8: Prompt used for the input-first approach of instance generation. The model is prompted to generate the instance first, and then generate the corresponding output. For instructions that don't require additional input, the output is allowed to be generated directly.

Attribution: <https://arxiv.org/pdf/2212.10560.pdf#page=16>
(<https://arxiv.org/pdf/2212.10560.pdf#page=16>)

Difficulties in Generating the Input/Output part: Classification tasks

Is Synthetic Data generation with an LLM really so easy ?

Although the few-shot learning approach to generating an Input/Output given an Instruction

- seems straightforward
- the authors encountered difficulties when generating Input/Output for Classification tasks

Consider the an Instruction Following example for a Classification task

Task: Classify the sentiment of the sentence into positive, negative, or mixed

Example 1

Sentence: I enjoy the flavor of the restaurant but their service is too slow.

Class Label: mixed

The authors found that the Response (i.e., Class Label) part generated by the LLM for Classification

- were examples whose Class Label's
- were *not well-distributed* among all possible labels
 - examples with certain labels were either over or under represented

This issue was traced

- to the **format** of the exemplars

using a deeper understanding of the Language Modeling task.

The original *format* of an exemplar used *Input-first* format

$$\langle \text{Instruction}^{(i)}, \text{Context}^{(i)}, \text{Response}^{(i)} \rangle$$

That is: the Response part came **last**.

By changing the format to

$$\langle \text{Instruction}^{(i)}, \text{Response}^{(i)}, \text{Context}^{(i)} \rangle$$

that is: placing the Response in the middle

- the distribution of Responses was improved

For example:

Task: Classify the sentiment of the sentence into positive, negative, or mixed

Example 1

Class Label: mixed

Sentence: I enjoy the flavor of the restaurant but their service is too slow.

Example 2

Class label: Positive

Sentence: I had a great day today. The weather was beautiful and I spent time with friends.

This is an example of Prompt Engineering

- In-context learning seems very sensitive to the format of prompts
- There is a skill of engineering a prompt to elicit the desired behavior

This feels similar to the idea behind Chain of Thought prompting

- by presenting `Class Label` first
- the model seems better conditioned to generate a less biased distribution of labels

Generating Instructions via Backtranslation

We now present an alternate method for generating the Instruction part of
⟨Instruction⟩

of an Instruction Following example

⟨Instruction, Context, Response⟩

The method is called *Back Translation*.

Given an LLM M_{xy}

- trained on $\langle \backslash \mathbf{x}, \backslash \mathbf{y} \rangle$ pairs
=
 $\langle \text{Instruction},$
 $\text{Response} \rangle$

we will train an "inverse" LLM M_{yz}

- trained on $\langle \backslash \mathbf{y}, \backslash \mathbf{x} \rangle = \langle \text{Response}, \text{pairs}$
 $\text{Instruction} \rangle$
- obtained from the training data for M_{xy}

This seems odd at first glance.

But M_{yz}

- can take a target y
- and generate a *synthetic* feature vector y

For the goal of creating synthetic Instruction Following examples:

M_{xy} is an LLM that generates

Response

given input

$\langle \text{Instruction, Context} \rangle$

M_{yz} will generate

$\langle \text{Instruction, Context} \rangle$

given input

Response

This is similar in spirit to the Language Modeling task

- we take an abundant source of unlabeled data
 - documents for the LLM
 - "answers" for the Synthetic Instruction Following task
- and create targets/labels
 - the continuation of a prefix for the LLM
 - the $\langle \text{Instruction}, \text{Context} \rangle$ for the Synthetic Instruction Following task

The advantage of this approach is that

- un-labeled data is plentiful
 - almost any block of text
- but labeled data (Response/Instruction pairs) is scarce.

So, starting with a plentiful resource, we create the scarce resource

- i.e, Instruction Following example triplet

Self-Improvement

We have seen [Self-Improvement \(LLM_Self_Improvement.ipynb\)](#) before

- Fine-tuning an LLM in stages

Recall:

- train the Target model $\backslash \text{model}$ in stages
 - creating a sequence of fine-tuned Target models $\backslash \text{model}_{(0)}, \backslash \text{model}_{(1)}, \dots$
 - of increasing power
- base case
 - fine-tune initial Target $\backslash \text{model}_{(0)}$
 - using a mixture of strong (human-generated) and weak (LLM generated) fine-tuning examples of the Target task
 - resulting in weak Target model $\backslash \text{model}_{(1)}$
- inductive case
 - create improved Target $\backslash \text{model}_{(+1)}$
 - by fine-tuning $\backslash \text{model}_{\backslash \text{tp}}$
 - with the strong examples we already have
 - augmented with examples created as outputs of Target model $\backslash \text{model}_{\backslash \text{tp}}$

For the purpose of generating Synthetic Instruction Following examples:

With the newly extended set of seed Instruction/Response pairs

- we have more exemplars
- which we can use as a seed to another iteration of M_{yz}
 - the enlarged set of exemplars may result in *better* synthetic Instruction/Response pairs

We can iterate on this process multiple times

- using the Augmented set of Instruction/Response pairs from step i
- as the "seed" for iteration $(i + 1)$ of the process

Here is the workflow:

Instruction Backtranslation

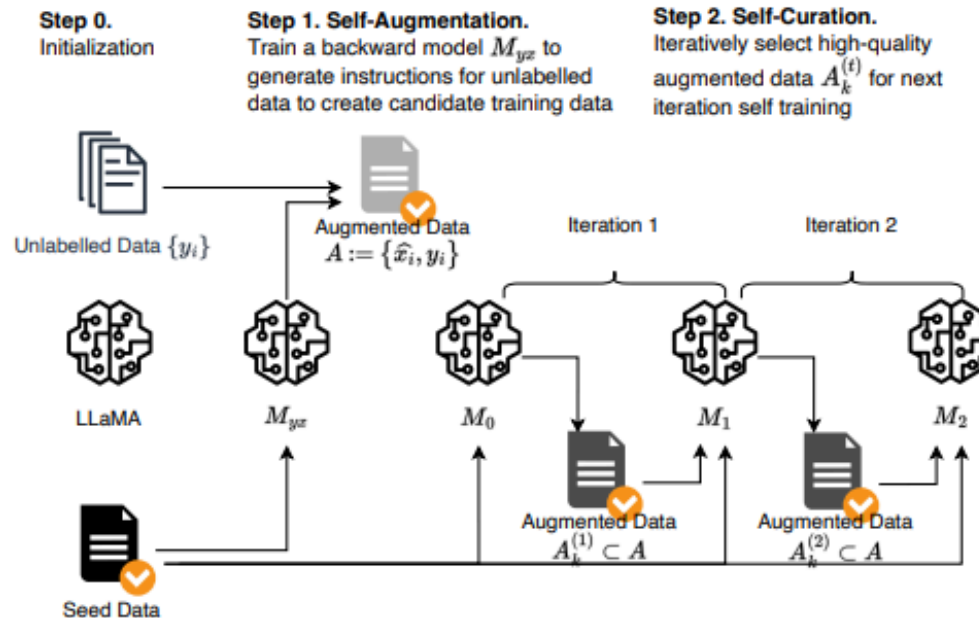


Figure 1: An overview of our **instruction backtranslation** method. We start from a base language model, e.g. LLaMa, a small amount of seed examples of (instruction, output) pairs, and a collection of unlabelled documents which are considered candidate outputs for unknown instructions. **Self-augmentation:** the base model is finetuned with (output, instruction) pairs from the seed examples as an instruction prediction model M_{yx} , which is used to generate candidate instructions for outputs from the unlabelled data. **Self-curation:** starting from an intermediate instruction-following model M_0 finetuned from seed examples only, it selects high-quality (instruction, output) pairs $A_k^{(1)}$ from the candidates from the previous step, and uses them as finetuning data for the next intermediate model M_1 , which is in turn used to select training data for obtaining M_2 .

Attribution: <https://arxiv.org/pdf/2308.06259.pdf#page=2>

Selecting the best synthetic examples for augmentation

The quality of the synthetic examples created at each step may not be uniformly high.

It would be desirable

- to select only the best examples to use
- in augmenting the seed examples of each iterative Step.

How can we rate the quality of a synthetic example ?

Ask the LLM to do it for you !

Using just the seed data

- fine tune a "first generation" LLM
 - denoted M_0
- to create a quality score of examples

The following prompt requests that the LLM evaluate the synthetic example using a rating scale of 1 (low quality) to 5 (high quality)

Instruction Backtranslation Curation

Below is an instruction from an user and a candidate answer. Evaluate whether or not the answer is a good example of how AI Assistant should respond to the user's instruction. Please assign a score using the following 5-point scale:

1: It means the answer is incomplete, vague, off-topic, controversial, or not

Use M_0 to

- select the best first generation augmented examples (from the first iteration)

The next generation augmented data set is

- the prior generation
- augmented with the best (highest quality scores) of the new generation

Now that we have

- an augmented (high quality) "generation i " set of seed examples

we continue our iterative process

- creating a more powerful scoring LLM M_i
- using exemplars
 - instructions
 - with scores from the generation $(i - 1)$ scorer M_{i-1}

The scores of M_i can then be used to create

- an even higher quality scorer M_{i+1}

This too is an example of [LLM Self-Improvement \(LLM_Self_Improvement.ipynb\)](#).

- improving the Scoring LLM

In [2]: `print("Done")`

Done

